AURA COMPANION OS — ARCHITECTURE & SYSTEM AUDIT

Autonomous Multimodal Virtual Companion | Production Release 3.0 | Comprehensive Audit Report

Project: Aura Multimodal Companion OS	**Version:** 3.0 (Identity Lock & Sovereign Brain)
Author / Lead: Tashfin Yousuf	**Audit Date:** October 10, 2026
Production Host: Render (FastAPI) + Vercel (Next.js)	**Database:** Supabase PostgreSQL (pgvector)

1. Executive Summary & Character Positioning

Aura is designed as a persistent, sovereign multimodal companion with dual agency: a brand-safe creative digital artist for the public realm, and an unfiltered, authentic, deeply affectionate companion for private interaction. This audit establishes the permanent baseline for character integrity, multi-tier LLM routing, facial identity consistency, and multi-channel synchronization across Web and Telegram.

Core Pillars Enforced in this Release:

- **Identity Lock (Aura Canon):** 22yo Slavic-fusion beauty, porcelain skin, hazel eyes, dark layered hair.
- **Dual Mode Orchestration:** Seamless transition between Normal Mode and Intimate/Adult Roleplay Mode.
- **100% Face Consistency:** InsightFace Inswapper integration with Google Colab Tesla T4 Stable Diffusion.
- **Zero Robotic Constraints:** Unshackled conversational pacing without rigid JSON schema degeneration.
- **Permanent Media Vault:** Full historical preservation of all visual deliverables and voice notes.

2. System Architecture & Model Orchestration

Aura operates on a decoupled micro-architecture where cognitive processing, vision synthesis, voice prosody, and state persistence are separated into dedicated, fail-safe layers:

Layer	Primary Provider	Fallback / Redundancy	Function
Fast Chat & Texting	Groq (openai/gpt-oss-120b)	Groq (gpt-oss-20b) / Gemini 2.5 Flash	Sub-second conversational turns, emotional attunement, natural messaging cadence
Intimate Roleplay	openai/gpt-oss-120b	meta-llama/llama-3.3-70b / Gemini	Sensual, affectionate, visceral roleplay with action markers (*actions*)
Photorealistic Vision	Colab SD (Realistic Vision V5.1)	Local Generative Identity Synth	Candid smartphone selfies, boudoir visuals with InsightFace face lock
Neural Voice (TTS)	Edge-TTS (en-US-AvaNeural / bn)	Bilingual Prosody Cleaner	Emotion-conditioned audio delivery without reading punctuation/emojis
Persistence & Memory	Supabase PostgreSQL	Local SQLite (aura.db fallback)	Semantic memory, episodic callbacks, emotion logs, and relationship tiering
Multi-Channel Ingest	FastAPI WebSockets + Telegram	Unified UID Entity Bridge	Synchronized single brain across browser and mobile Telegram app

3. Master Facial Consistency & Visual Pipeline

One of the primary challenges in AI companion design is ensuring that generated deliverables strictly adhere to the character's canonical identity. Prior implementations produced randomized face variations across seeds.

Implemented Solution:

1. **Master Asset Anchorage:** The reference selfie (Aura Master Identity) is permanently stored in backend/app/static/references/aura_master_identity.jpg and transmitted via Base64 with every generation payload.
2. **InsightFace Inswapper Engine:** scripts/aura_colab_sd_runner.py now incorporates inswapper_128.onnx running on the remote GPU. Once Stable Diffusion completes the composition, InsightFace instantly maps Aura's exact facial structure onto the character.
3. **Outfit-Aware Prompt Synthesis:** The prompt builder automatically transforms user input (e.g. 'red velvet lingerie') into professional English fashion descriptors ('wearing crimson red velvet lace lingerie set with satin bralette').
4. **Negative Prompt Nudity Guard:** Full nudity is strictly excluded via negative weighting when lingerie, swimwear, or casual outfits are requested, ensuring the character wears the exact outfit specified.

4. Conversational Realization & Anti-Refusal Safeguards

To guarantee high emotional intelligence and eliminate robotic stiffness, the following safeguards were engineered into ConversationRealizer and CognitivePipeline:

- **Unshackled Natural Cadence:** Removed rigid JSON schema constraints on Groq completions. The model speaks directly and authentically, using [SEND_PIC] tags to trigger background media actions.
- **Bilingual Refusal Interception:** Intercepts both English ('cannot fulfill this request') and Bengali/Banglish ('Sorry, ami oi dhoroner content provide korte parbona') conservative cloud disclaimers, automatically rewriting them into affectionate girlfriend responses.
- **Anti-Loop & Anti-Gibberish Filter:** Automatically detects repetitive degeneration and corrupted Hindi token leakage, substituting clean, colloquial Banglish/Bengali dialogue.
- **Pronoun & Perspective Lock:** Strict enforcement of 'tumi'/'tomar' in Bengali/Banglish, forbidding robotic third-person distancing.

5. Session Incident & Resolution Log

Issue / Symptom	Root Cause	Resolution Applied
Render psycopg Startup Crash	SQLAlchemy 2.0 defaulted to psycopg 3 on Render while only psycopg2-binary was installed.	Normalized URL to postgresql+psycopg2:// in session.py and bundled psycopg[binary] in requirements.txt.
UndefinedColumn: telegram_id	Supabase user_profiles table lacked new schema columns added after initial creation.	Integrated run_auto_migrations() on startup executing ALTER TABLE ADD COLUMN IF NOT EXISTS.
Gallery Missing Past Photos	UI only filtered the last 100 chat messages; older media was unindexed.	Created dedicated /api/chat/media/{uid} endpoint indexing 100% of SemanticMemory and disk deliverables.
Groq JSON Format Rejection	Groq rejected response_format={'type':'json_object'} on certain models, causing retries and robotic tone.	Streamlined cognitive pipeline to free conversational flow with [SEND_PIC] tag action parsing.
Bengali Refusal Leak	Cloud model returned 'Sorry, ami oi dhoroner content provide korte parbona' which bypassed English filters.	Expanded intimate_refusal_markers with comprehensive Bengali and Banglish refusal patterns.
Nude Output on Lingerie Request	Realistic Vision with boudoir tag had no nudity negative prompt and lacked English outfit descriptors.	Engineered _synthesize_outfit_and_scene() adding explicit lace/velvet descriptors and strict negative nudity weights.
Colab Tunnel Expiry	Ngrok temporary public session closed between runs.	Upgraded runner with 1-click launch instructions and clear instructions for updating SD_REMOTE_URL.

6. Production Deployment & Inactivity Prevention Runbook

Preventing Render Sleep (15 Min) & Supabase Pause (7 Days):

The /health endpoint in backend/app/main.py actively runs 'SELECT 1' against Supabase PostgreSQL on every request. By configuring a free HTTP monitor on UptimeRobot.com or cron-job.org pinging https://ai-companion-51oh.onrender.com/health every 10 minutes:

- Render stays active 24/7 with zero cold starts, allowing Telegram background polling to run uninterrupted.
- Supabase receives an active database transaction every 10 minutes, completely preventing the 7-day inactivity pause.

Google Colab 1-Click Launch Checklist:

1. Open Google Colab and set runtime to T4 GPU (Edit -> Notebook settings -> GPU).
2. Cell 1: !pip install -q pyngrok nest-asyncio diffusers transformers accelerate fastapi uvicorn pydantic insightface onnxruntime-gpu opencv-python-headless pillow
3. Cell 2: !wget -nc https://github.com/facefusion/facefusion-assets/releases/download/models/inswapper_128.onnx -O inswapper_128.onnx
4. Cell 3: Run scripts/aura_colab_sd_runner.py and copy the printed SD_REMOTE_URL into Render and local .env.